

Jogo de Azulejos

Barchin e Charos estão jogando um jogo em um tabuleiro de células quadradas unitárias de $2N \times 2M$. As linhas são numeradas 0 a $2N - 1$ de cima para baixo, e as colunas são numeradas 0 a $2M - 1$ da esquerda para a direita. Para $0 \leq i < 2N$ e $0 \leq j < 2M$, denotamos a célula na linha i e coluna j por (i, j) .

Barchin fornece a Charos $N \cdot M$ **blocos**, um por um. Cada bloco é um quadrado 2×2 composto por quatro **peças** de 1×1 . Barchin coloriu cada peça de cada bloco de preto ou branco, garantindo que **pelo menos uma peça seja branca**.

Charos deve colocar cada bloco no tabuleiro **imediatamente após recebê-lo**, sem saber quais blocos receberá posteriormente. Os blocos não podem ser rotacionados. Cada bloco deve ser colocado inteiramente dentro do tabuleiro, cobrindo exatamente quatro células. Além disso, a célula superior esquerda de cada bloco deve cobrir uma célula cujas coordenadas de linha e coluna sejam ambas pares. Cada célula do tabuleiro pode ser coberta por, no máximo, um bloco.

Barchin vence o jogo se, após a colocação de qualquer bloco, existir um quadrado de células 2×2 coberto por quatro peças pretas. Formalmente, se as células (a, b) , $(a + 1, b)$, $(a, b + 1)$, $(a + 1, b + 1)$ estiverem todas cobertas por peças pretas para algum $0 \leq a < 2N - 1$ e $0 \leq b < 2M - 1$, então Barchin vence. Os índices a e b não precisam ser pares.

Charos vence se ela colocar todos os blocos $N \cdot M$ sem que Barchin vença nenhuma vez. Observe que a colocação de $N \cdot M$ blocos cobrirá completamente o tabuleiro.

Sua tarefa é implementar uma estratégia para que Charos vença o jogo. Pode-se provar que, sob as restrições dadas, Charos sempre pode posicionar os blocos de forma a garantir a vitória, independentemente das cores dos blocos que ela receber posteriormente.

Detalhes de Implementação

Você deve implementar dois procedimentos:

```
void init(int N, int M);
```

- N : metade do número de linhas no tabuleiro.
- M : metade do número de colunas no tabuleiro.

- O procedimento é chamado exatamente uma vez por caso de teste, no início da execução do seu programa.

```
std::pair<int, int> receive_block(int TL, int TR, int BL, int BR);
```

- TL, TR, BL, BR : as cores dos ladrilhos superior esquerdo, superior direito, inferior esquerdo e inferior direito do bloco atual, respectivamente, conforme mostrado na figura abaixo. Cada valor é 0 (branco) ou 1 (preto).
- Este procedimento é chamado exatamente $N \cdot M$ vezes por caso de teste, após a chamada inicial a `init`.



Este procedimento deve retornar um par de inteiros (i, j) , onde i é a coordenada da linha e j é a coordenada da coluna da célula onde o ladrilho superior esquerdo deste bloco deve ser colocado. Tanto i quanto j devem ser pares, e a região 2×2 coberta pelo bloco não deve sobrepor nenhum bloco colocado anteriormente.

Se `receive_block` alguma vez retornar um par que não satisfaça esses requisitos, ou se depois de colocar o bloco um quadrado de células de 2×2 ficar completamente coberto por ladrilhos pretos, o corretor encerrará imediatamente seu programa e o veredito para o caso de teste será `Output isn't correct`.

O comportamento do corretor **não é adaptativo**. Isso significa que a sequência de blocos que Barchin fornece a Charos é fixada antes da chamada de `init`.

Restrições

Para cada bloco, seja S o número de peças pretas entre suas quatro peças. Ou seja, $S = TL + TR + BL + BR$.

- $1 \leq N, M \leq 100$
- $0 \leq S \leq 3$ para cada bloco.

Subtarefas

Subtarefa	Pontuação	Restrições adicionais
1	6	$S = 1$ para cada bloco e $N = 2$.
2	16	$S = 3$ para cada bloco. $N = M$, N é par e cada uma das quatro colorações de bloco possíveis aparece exatamente $\frac{N^2}{4}$ vezes.
3	10	$S = 1$ por bloco.
4	29	$S \leq 2$ por bloco.
5	39	Sem restrições adicionais.

Exemplo

Considere um jogo com $N = 1$ e $M = 2$, então o tabuleiro tem 2 linhas e 4 colunas. O corretor primeiro faz a chamada:

```
init(1, 2)
```

Inicialmente, todas as células estão vazias. O tabuleiro tem a seguinte aparência:

	0	1	2	3
0				
1				

Existem $N \cdot M = 2$ blocos para colocar. Suponha que Barchin forneça um bloco com três peças pretas e uma peça branca no canto superior direito. O corretor faz a chamada:

```
receive_block(1, 0, 1, 1)
```

Charos decide colocar este bloco no lado esquerdo do tabuleiro retornando $(0, 0)$.

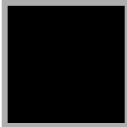
O tabuleiro agora está assim:

	0	1	2	3
0				
1				

Em seguida, Barchin apresenta outro bloco com três peças pretas e uma peça branca no canto superior esquerdo:

```
receive_block(0, 1, 1, 1)
```

A única célula restante com número par de linhas e colunas que pode servir como canto superior esquerdo de um bloco 2×2 é $(0, 2)$, portanto Charos retorna $(0, 2)$. O tabuleiro fica assim:

	0	1	2	3
0				
1				

Nenhum quadrado 2×2 está completamente coberto por peças pretas, então Charos conseguiu posicionar todos os blocos sem que Barchin vencesse. Charos ganha o jogo.

Corretor exemplo

Formato de entrada:

```
N M
TL[0] TR[0] BL[0] BR[0]
TL[1] TR[1] BL[1] BR[1]
...
TL[NM-1] TR[NM-1] BL[NM-1] BR[NM-1]
```

Formato de saída:

```
R[0] C[0]  
R[1] C[1]  
...  
R[NM-1] C[NM-1]
```

Aqui, $R[k]$ e $C[k]$ são o par de inteiros retornado pela k -ésima chamada a `receive_block`.